

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Защита программного обеспечения с ограничением пробного периода»

Выполнили:

Суханкулиев Мухаммет,
студент группы N3346



(подпись)

Проверил:

Ищенко Алексей Петрович,
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение	3
1 Программа учета пользователей (Демо-версия)	4
1.1 Задание	4
1.2 Теоретическая часть	4
1.2.1 Хранение данных для защиты от переустановки	4
1.2.2 Механизмы ограничения пробного периода	5
1.2.3 Инструменты сборки и дистрибуции	5
1.3 Практическая реализация	6
1.3.1 Архитектура приложения	6
1.3.2 Реализация системы защиты	6
1.3.3 Сборка инсталлятора	7
1.4 «Взлом» программы	7
Заключение	9
Список использованных источников	10

ВВЕДЕНИЕ

Цель работы – разработка и исследование методов защиты программного обеспечения от несанкционированного использования на примере создания утилиты с ограниченным пробным периодом.

Для достижения поставленной цели необходимо разработать:

- консольное приложение с основной бизнес-логикой (ввод и сохранение данных);
- гибридную систему лицензирования, ограничивающую работу программы по времени и по количеству запусков;
- механизм сохранения состояния лицензии, устойчивый к простому удалению и повторной установке программы;
- пакет для дистрибуции (инсталлятор и деинсталлятор), который корректно регистрирует приложение в операционной системе.

1 ПРОГРАММА УЧЕТА ПОЛЬЗОВАТЕЛЕЙ (ДЕМО-ВЕРСИЯ)

1.1 Задание

Разработать программу, которая запрашивает ФИО пользователя и сохраняет эту информацию в текстовый файл. Если такое ФИО уже существует, программа должна выводить соответствующее сообщение.

Программа должна быть оснащена системой защиты, ограничивающей ее использование в рамках пробного периода. Реализованы две версии ограничений, действующие одновременно:

- **Time-limited:** ограничение по времени использования (3 минуты с момента первого запуска).
- **Start-limited:** ограничение по количеству запусков (5 запусков).
- По достижении любого из лимитов программа должна блокировать свой основной функционал и предлагать пользователю приобрести полную версию. Система защиты должна быть персистентной, то есть сохранять свое состояние (оставшееся время и количество запусков) даже после деинсталляции и повторной установки программы.

Итоговый продукт должен включать инсталлятор, саму программу и деинсталлятор.

1.2 Теоретическая часть

1.2.1 Хранение данных для защиты от переустановки

Ключевой задачей при реализации пробной версии является сохранение данных о лицензии таким образом, чтобы они не удалялись вместе с программой. Простое хранение данных в папке с программой неэффективно, так как эта папка удаляется деинсталлятором.

Оптимальным решением в ОС Windows является использование реестра Windows. **Реестр** – это иерархическая база данных для хранения настроек операционной системы и приложений. Данные в реестре не привязаны к файлам программы и по умолчанию не удаляются при деинсталляции.

В данной работе для хранения данных о триале (счетчик запусков, дата первой установки) используется ветка `HKEY_CURRENT_USER\Software\Lab2App`.

- `HKEY_CURRENT_USER` (HKCU): этот куст хранит настройки для текущего вошедшего в систему пользователя. Данные, сохраненные здесь, будут уникальны для каждого пользователя компьютера.

- Работу с реестром из Python обеспечивает встроенный модуль `winreg`.

Пользовательские данные (файл с ФИО) хранятся в общей папке `C:\ProgramData\Lab2App`, что делает их доступными для всех пользователей системы и также защищает от удаления при деинсталляции.

1.2.2 Механизмы ограничения пробного периода

В программе реализована гибридная модель защиты, сочетающая два подхода:

1. Ограничение по количеству запусков (Start-limited):

- Принцип: в реестре создается значение (например, `RunCount` типа `REG_DWORD`), которое инкрементируется при каждом успешном запуске программы.
- Проверка: перед выполнением основной логики программа считывает текущее значение счетчика и сравнивает его с максимальным допустимым (`MAX_RUNS`). Если лимит превышен, работа блокируется.

2. Ограничение по времени (Time-limited):

- Принцип: при самом первом успешном запуске программы в реестр записывается точная временная метка (например, `InstallDate` типа `REG_SZ` в формате ISO).
- Проверка: при каждом последующем запуске программа считывает дату первого запуска, прибавляет к ней разрешенный временной интервал (`TRIAL_MINUTES`) и сравнивает полученную дату истечения срока с текущим системным временем. Если текущее время больше, работа блокируется.

Такой гибридный подход надежен, так как не позволяет пользователю обойти ограничение, например, просто не закрывая программу.

1.2.3 Инструменты сборки и дистрибуции

PyInstaller: утилита для преобразования скриптов Python (`.py`) в самостоятельные исполняемые файлы (`.exe`) для Windows. Она собирает все необходимые зависимости и сам интерпретатор Python в один файл или папку, что позволяет запускать программу на компьютерах, где Python не установлен.

Inno Setup: мощная и гибкая система для создания инсталляторов для Windows. Она позволяет с помощью простого скрипта (`.iss`) определить все шаги установки: копирование файлов, создание ярлыков в меню «Пуск» и на рабочем столе, запись ключей

в реестр и, что важно, автоматическое создание деинсталлятора, который корректно удаляет программу из системы.

1.3 Практическая реализация

1.3.1 Архитектура приложения

Программа написана на языке Python 3 и имеет модульную структуру, разделяющую логику работы с лицензией и основную функциональность.

- `get_license_data()`: функция для чтения данных о лицензии из реестра. Обработывает случай первого запуска (когда ключ в реестре еще не создан).
- `check_license()`: проверяет, не истекли ли лимиты, и выводит пользователю информацию о текущем состоянии триала.
- `commit_license_use()`: записывает в реестр обновленный счетчик запусков и, при необходимости, дату первого запуска. Вызывается только после успешного выполнения основной логики.
- `run_core_logic()`: реализует основной функционал – пошаговый ввод ФИО, валидацию, проверку на дубликаты и запись в файл `C:\ProgramData\Lab2App\users.txt`.
- `main()`: главная управляющая функция, определяющая порядок вызовов.

1.3.2 Реализация системы защиты

Система защиты реализована путем сохранения двух значений в реестре по пути `HKEY_CURRENT_USER\Software\Lab2App`:

- `RunCount` (тип `REG_DWORD`): хранит количество запусков.
- `InstallDate` (тип `REG_SZ`): хранит дату и время первого успешного запуска в формате ISO 8601.

При каждом запуске программа выполняет следующую последовательность действий:

1. Читает данные из реестра.
2. Проверяет, не превышены ли лимиты. Если да – завершает работу.
3. Если лимиты не превышены, выполняет основную логику.
4. Если основная логика выполнена успешно, обновляет счетчик запусков в реестре.

Такой подход гарантирует, что неудачные или прерванные запуски не будут уменьшать количество доступных попыток.

1.3.3 Сборка инсталлятора

Для создания инсталлятора используется скрипт `setup.iss` (созданный автоматически) и компилятор Inno Setup. Скрипт выполняет следующие действия:

1. Определяет метаданные приложения (название, версия, разработчик).
2. Указывает путь к собранному `main.exe` файлу.
3. Настраивает установку в папку Program Files.
4. Создает ярлыки в меню «Пуск» и (опционально) на рабочем столе.
5. Регистрирует деинсталлятор в разделе «Установка и удаление программ» Windows.

Важно, что скрипт инсталлятора **не содержит** инструкций по удалению ключа реестра `HKEY_CURRENT_USER\Software\Lab2App`, что и обеспечивает персистентность защиты.

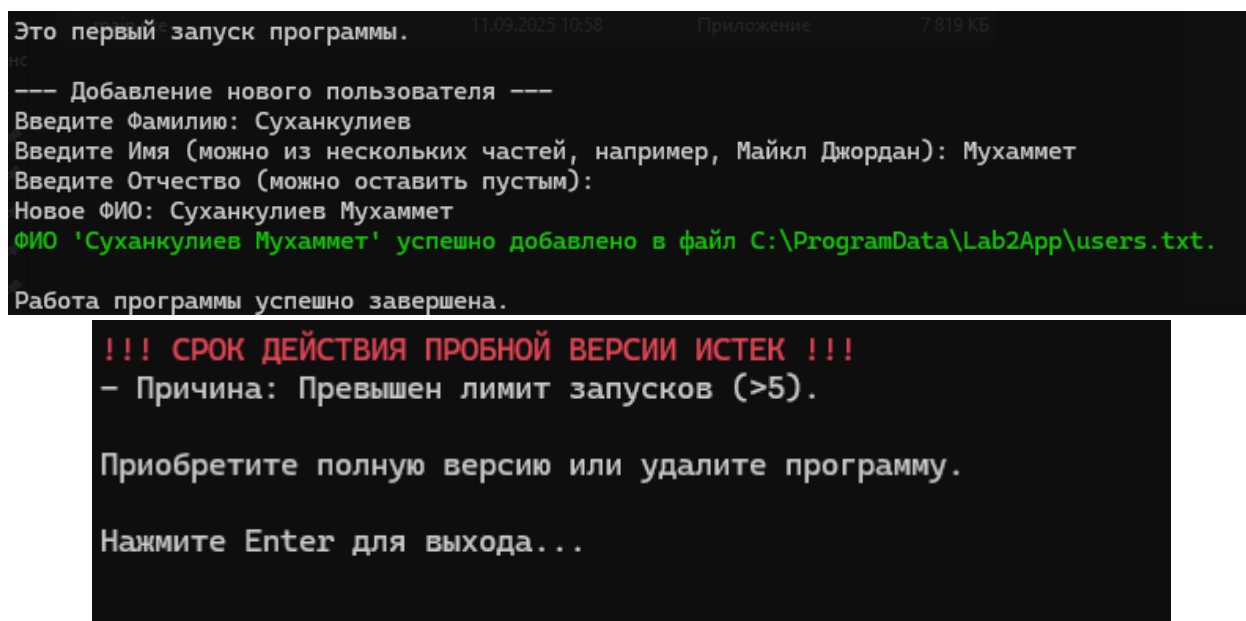


Рисунок 1 – Примеры работы программы

1.4 «Взлом» программы

В рамках задания требовалось изучить способ обхода реализованной защиты. Так как данные о триале хранятся в известном месте в реестре, «взлом» сводится к удалению этого ключа.

Инструкция по «взлому»:

1. Открыть редактор реестра (`regedit.exe`).
2. Перейти по пути `HKEY_CURRENT_USER\Software\Lab2App`.
3. Удалить всю папку (ключ) `Lab2App`.

После выполнения этих действий программа при следующем запуске не найдет данных о предыдущих установках и начнет пробный период заново.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно разработана программа с гибридной системой защиты, ограничивающей использование по времени и количеству запусков. Был реализован механизм персистентности, использующий реестр Windows для хранения состояния лицензии, который устойчив к переустановке приложения.

Это позволило на практике изучить методы хранения персистентных данных, принципы работы пробных версий программного обеспечения, а также освоить инструменты для создания полноценного дистрибутива, включая исполняемый файл и инсталлятор. Цель работы была достигнута в полном объеме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ищенко А. П. Технологии и методы программирования – Лекция 1: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
2. Ищенко А. П. Технологии программирования: жизненный цикл и разработка ПО – Лекция 2: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
3. Работа с реестром Windows в Python : техническая документация. – Текст : электронный. – 2025. – URL : <https://docs.python.org/3/library/winreg.html>
4. Inno Setup Documentation : техническая документация. – Текст : электронный. – 2025. – URL : <https://jrsoftware.org/ishelp/>
5. Русинович М. Внутреннее устройство Microsoft Windows / М. Русинович, Д. Соломон, А. Ионеску. – 6-е изд. – Санкт-Петербург : Питер, 2013. – 800 с. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг А.1 – lab_app.py

```
import os
import sys
import winreg
from datetime import datetime, timedelta

# --- КОНСТАНТЫ И НАСТРОЙКИ ---

# Храним данные в общей папке ProgramData, чтобы они были доступны всем
# пользователям и не удалялись при деинсталляции программы.
APP_DATA_DIR = os.path.join(
    os.environ.get("ALLUSERSPROFILE", "C:\\\\ProgramData"), "Lab2App"
)
DATA_FILE_NAME = os.path.join(APP_DATA_DIR, "users.txt")

# Параметры пробной версии
MAX_RUNS = 5
TRIAL_MINUTES = 3

# Уникальный путь в реестре для хранения данных о запусках и дате установки.
REG_KEY_PATH = r"Software\Lab2App"

def get_license_data():
    """Читает данные о лицензии (счетчик запусков, дата установки) из
    реестра."""
    try:
        key = winreg.OpenKey(winreg.HKEY_CURRENT_USER, REG_KEY_PATH, 0,
            winreg.KEY_READ)
        run_count, _ = winreg.QueryValueEx(key, "RunCount")
        install_date_str, _ = winreg.QueryValueEx(key, "InstallDate")
        winreg.CloseKey(key)
        return int(run_count), datetime.fromisoformat(install_date_str)
    except FileNotFoundError:
        # Если ключ не найден, значит это первый запуск.
        return 0, None
    except Exception as e:
        # При любой другой ошибке чтения блокируем программу для
        безопасности.
        print(f"Ошибка чтения данных лицензии: {e}")
        return MAX_RUNS + 1, datetime.now() - timedelta(days=1)

def check_license(run_count, install_date):
    """Проверяет, не истекли ли лимиты по времени или по количеству
    запусков."""
    if install_date is None:
        print("Это первый запуск программы.")
        return True

    expiry_date = install_date + timedelta(minutes=TRIAL_MINUTES)
    time_limit_exceeded = datetime.now() > expiry_date
    run_limit_exceeded = (run_count + 1) > MAX_RUNS

    if time_limit_exceeded or run_limit_exceeded:
```

```

        print("\033[91m" + "!!! СРОК ДЕЙСТВИЯ ПРОБНОЙ ВЕРСИИ ИСТЕК !!!" +
"\033[0m")
        if time_limit_exceeded:
            print(
                f"- Причина: Превышен лимит времени использования
({TRIAL_MINUTES} min).")
            )
        if run_limit_exceeded:
            print(f"- Причина: Превышен лимит запусков (>{MAX_RUNS}).")

        print("\nПриобретите полную версию или удалите программу.")
        return False
    else:
        time_left = expiry_date - datetime.now()
        print("\033[96m" + "--- Информация о пробной версии ---")
        print(
            f"Запусков использовано: {run_count} из {MAX_RUNS}. Этот запуск
№{run_count + 1}."
        )
        minutes_left = time_left.seconds // 60
        seconds_left = time_left.seconds % 60
        print(f"Времени осталось: {minutes_left} мин {seconds_left} сек.")
        print("-----" + "\033[0m")
        return True

def commit_license_use(run_count, install_date):
    """Записывает в реестр обновленный счетчик запусков."""
    try:
        key = winreg.CreateKey(winreg.HKEY_CURRENT_USER, REG_KEY_PATH)
        if install_date is None:
            # При первом успешном запуске фиксируем дату установки.
            install_date = datetime.now()
            winreg.SetValueEx(
                key, "InstallDate", 0, winreg.REG_SZ,
install_date.isoformat()
            )
            winreg.SetValueEx(key, "RunCount", 0, winreg.REG_DWORD, run_count +
1)
            winreg.CloseKey(key)
        except Exception as e:
            print(f"\033[91mОшибка при обновлении данных лицензии: {e}\033[0m")

def run_core_logic():
    """Основная логика: пошаговый ввод ФИО и запись в общий файл."""
    try:
        print("\n--- Добавление нового пользователя ---")

        # --- Пошаговый ввод и валидация ---
        while True:
            last_name = input("Введите Фамилию: ").strip()
            if last_name and all(part.isalpha() for part in
last_name.split("-")):
                break
            print(
                "\033[93mОшибка: Фамилия не может быть пустой и должна
содержать только буквы/дефисы.\033[0m"
            )

        while True:
            first_name = input(

```

```

        "Введите Имя (можно из нескольких частей, например, Майкл
Джордан): "
    ).strip()
    if first_name and all(
        part.isalpha() for part in first_name.replace("-", "
").split()
    ):
        break
    print(
        "\033[93mОшибка: Имя не может быть пустым и должно содержать
только буквы/дефисы.\033[0m"
    )

    middle_name = input("Введите Отчество (можно оставить пустым):
").strip()
    if middle_name and not all(part.isalpha() for part in
middle_name.split("-")):
        print(
            "\033[93mПредупреждение: Отчество содержит недопустимые
символы. Оно будет проигнорировано.\033[0m"
        )
        middle_name = ""

    # Корректная обработка заглавных букв в составных именах ("анна-
мария" -> "Анна-Мария").
    last_name_capitalized = "-".join(
        [part.capitalize() for part in last_name.split("-")]
    )
    first_name_capitalized = " ".join(
        [
            "-".join([p.capitalize() for p in part.split("-")])
            for part in first_name.split()
        ]
    )
    middle_name_capitalized = "-".join(
        [part.capitalize() for part in middle_name.split("-")]
    )

    full_name_parts = [last_name_capitalized, first_name_capitalized]
    if middle_name_capitalized:
        full_name_parts.append(middle_name_capitalized)

    full_name = " ".join(full_name_parts)
    print(f"Новое ФИО: {full_name}")

    # --- Работа с файлом данных ---

    # ВАЖНО: Запись в C:\ProgramData требует запуска программы от имени
администратора.
    try:
        os.makedirs(APP_DATA_DIR, exist_ok=True)
    except PermissionError:
        print(
            "\033[91mКритическая ошибка: Недостаточно прав для создания
директории в C:\\ProgramData."
        )
        print("Запустите программу от имени администратора.\033[0m")
        return False

    if os.path.exists(DATA_FILE_NAME):
        with open(DATA_FILE_NAME, "r", encoding="utf-8") as f:
            lines = [line.strip() for line in f.readlines()]

```

```

        if full_name in lines:
            print(
                "\033[93m"
                + f"Пользователь '{full_name}' уже существует в файле."
                + "\033[0m"
            )
            return True

    with open(DATA_FILE_NAME, "a", encoding="utf-8") as f:
        f.write(full_name + "\n")

    print(
        "\033[92m"
        + f"ФИО '{full_name}' успешно добавлено в файл {DATA_FILE_NAME}."
        + "\033[0m"
    )
    return True

except Exception as e:
    print("\033[91m" + f"Ошибка при работе с файлом: {e}" + "\033[0m")
    return False

def main():
    """Главная функция, управляющая логикой приложения."""
    run_count, install_date = get_license_data()

    if not check_license(run_count, install_date):
        input("\nНажмите Enter для выхода...")
        sys.exit()

    success = run_core_logic()

    if success:
        # Засчитываем запуск только если основная логика выполнена успешно.
        commit_license_use(run_count, install_date)
        print("\nРабота программы успешно завершена.")
    else:
        print("\nРабота программы завершилась с ошибкой. Запуск не будет засчитан.")

    input("Нажмите Enter для выхода...")

if __name__ == "__main__": # pragma: no cover
    main()

```